

zine

source

1. Philosophy and sheaf theory	3
2. Universal dependencies	4
3. Nanopass	5
3.1. L_0	5
3.2. L_1	6
4. Category theory	7
4.1. Definitions	7
4.2. A Dependency Tree Topos	7
5. Feeling the elephant	10
6. Diagrams	10
6.1. 3D	10
A. Referenced packages	11
B. Stubs	12
References	13

Zine is a human-computer interaction language for philosophy. It is interesting in that its syntax is natural people language; in fact, most human languages can be used. Nevertheless, it is precise.



Figure 1. Zines from the seventies.¹

Zine can also mean Zine Is Not Emacs, which I wrote this in.

Zine is also a very good what-you-see-is-what-you-get editor for Typst² technical writing.

With the widespread usage of LLMs the bullshit machine is running at full steam, and nuance is swept under probabalism.

If we contrapose the continental traditions with the analytical tradition, broadly taken to include mathematics all the natural sciences. In the analytical tradition, the rejection of accepting all statements as true is so strong that there is .

Even a mediocre engineer can contribute, while philosophy still takes enough discipline of the mind that prevention of falsehood is accomplished in the field through massive exclusion.

In physics, the tenth grade student can understand the Rutherford model of the atom and accept it as a fact upon which to build without

Science requires fixing a model first

Haskell is the glue, so that you can think about the parts in isolation. Category theory style Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

This resolves a nominal into a specific noun

resolve :: **Nominal** → **Noun**

newtype Dependency source target = Dependency (source → ([Rel], target))

Because Dependency is generic on any source or target, coreference can be handled separately on a PoS layer I bet

```
-instance Category Dependency where
- id = Dependency $ \source → ([], source)
- (Dependency a) . (Dependency b) = Dependency $ \source →
-   let (relations, intermediate) = b source
-       (relations', target) = a intermediate
-   in (relations ++ relations', target)
```

Dependency parsing happens sentence-by-sentence

type Syntax = [Sentence]

UDPipe (maybe see <https://github.com/fpco/inline-c>), lazily

dependencyParse :: **Text** → **Syntax**

Functor from the tree-category of Dependency to a meaningful category. See below Section 4.2 – use Prolog

understand :: **Syntax** → **Maybe Semantics**

explain :: **Semantics** → **Syntax**

Maxima here, which needs a ECL FFI or general Lisp IPC

simplify :: **Math** → **Math** – Maxima

Yi, The keybindings part

interpret :: **Input** → **Command**

Yi, stuff from EditorM for example

execute :: **Command** → **State** → **State**

Custom, using typst-layout, which needs rust FFI or some (??) IPC

view :: **State** → **DisplayList**

WebRender ofc, which needs rust FFI (see <https://github.com/harpocrates/inline-rust>) or general Rust IPC

webrender :: **DisplayList** → **GL**

```

typstLayout :: Typst.Syntax.Markup → DisplayList
createFrame :: IO GLContext
read :: IO Input

```

The main interaction loop

```

main :: IO ()
main = do
  frame ← createFrame
  let interactionLoop state = do
    display ((webrender . view) state) frame
    input ← read
    let state' = (execute . interpret) input state
    interactionLoop state'
  interactionLoop initialState

```

Profane definitions of types

```

newtype GL = GL { display :: GLContext → IO () }

```

Haskell default and glue:

- Prolog (NLP Topos FOL). Maybe this can be in native Haskell or scheme
- Lisp (CAS) Better interop with scheme
- Rust (rendering)
- C++ (text tagging, Firefox IPDL) Can be done with udpipes

lol maybe idris instead of haskell -> scheme -> rust marwood or other scheme implementation for display list, udpipes -> prolog with call/cc -> lisp direct interop with scheme lists but embedded via ecl2

if they're all separate, you could get a prolog repl, an ECL repl, and a GHCi repl

maybe prolog translates directly into lispy things

haskell -> prolog -> lisp

1. Philosophy and sheaf theory

For example, epistemology, at its most basic, can be considered

Fact $\longrightarrow$ Knowledge

Need to think about the pushout of rooted words, which would connect subjects to objects via some actual relation. Because of the categorical structure of Dependency, only the dependency paths between two different tokens matters. Pushouts can be related to specific morphisms, and will probably be through things like verbs

If I was to use sheaf theory here, then I've broken the problem into:

- Local semantics
- gluing, including coreference probably
- Global semantics

Globally, I want to extract stuff like

```

newtype Epistemology = Epistemology (Set Fact → Knowledge)

```

What is true about both the local semantics and the global semantics? Both subcategories of Hask, likely, as they both need to be representable internally by the computer ultimately.

So what is Epistemology? It's a subcategory with only two objects and it goes unidirectionally. pushouts are what two epistemic systems have in common and pullbacks are the dual

what about hegel

dialectic :: (Thesis, Thesis) → Maybe Thesis

metaphysics: what is? epistemology: what is knowledge? ethics: what is just/right/needful?

probably can't constrain this too much – maybe it's literally just collections of morphisms between two Hask objects

type Philosophy a b = [a → b]

ok now we might be getting somewhere, as I can also make morphisms from local semantics via pushouts through a tree root in the dependency parse. I don't necessarily have to ask from whence nouns came from (this is restricted to humans), just the relationships defined between them.

2. Universal dependencies

These are all the thirty-seven relations in the Universal Dependencies grammar³.

I'm currently just considering the input as CoNLL-U, wherever it comes from. For English and other well-resourced, UDLPipe 1 is probably sufficient.

This version keeps reloading the model, which is slow. I'm thinking I should keep a global cache of the models and never unload them, and manage that from C++. Also, errors should get propagated up somehow; right now, they just get printed out to the host process' standard error.

```
{-# NOINLINE udPipe #-}
udPipe :: FilePath → String → String
udPipe modelPath sentence = unsafePerformIO do
  withCString modelPath \cModel → withCString sentence \cSentence → do
    output ← [Cpp.exp| char* { [&]() -> char* {
      using namespace ufal::udpipe;

      model* m = model::load($(char* cModel));
      if (!m) {
        return strdup("");
      }

      pipeline p(
        m,
        "tokenizer=ranges",
        pipeline::DEFAULT,
        pipeline::DEFAULT,
        "conllu"
      );

      std::istringstream input($(char* cSentence));
      std::ostringstream output;
      std::string error;
```

```

    bool succeeded = p.process(input, output, error);
    if (!succeeded) {
        std::cerr << error << '\n';
    }

    delete m;
    return strdup(succeeded ? output.str().c_str() : "");
}() } ||
result ← peekCString output
free output
pure result

```

```

parse :: FilePath → String → Either String CoNLLU.Doc
parse modelPath = parseConllu "" . udPipe modelPath

```

3. Nanopass

Kent Dyvbig’s *nanopass* framework⁴. I use Marseille Bouchard Demko’s *nanopass* Haskell implementation here.

3.1. L_0

Let’s begin by defining

```

module L0 where

import Language.Nanopass

[deflang]
((UD w)
 (DependencyTree (DependencyTree Dependency UPOS w (* DependencyTree)))
 (Dependency
  (Ref)
  (Acl) (Advcl) (Advmod) (Amod) (Appos) (Aux)
  (Case) (Ccomp) (Cc) (Clf) (Compound) (Conj) (Cop) (Csubj)
  (Dep) (Det) (Discourse) (Dislocated)
  (Expl)
  (Fixed) (Flat)
  (Goeswith)
  (Iobj)
  (List)
  (Mark)
  (Nmod) (Nsubj) (Nummod)
  (Obj) (Obl) (Orphan)
  (Parataxis) (Punct)
  (Reparandum) (Root)
  (Vocative)
  (Xcomp))
 (UPOS
  (ADJ) (ADP) (ADV) (AUX)
  (CCONJ)
  (DET)

```

```

(INTJ)
(NOUN) (NUM)
(PART) (PRON) (PROPN) (PUNCT)
(SONJ) (SYM)
(VERB)
(X)))]

```

This language is a tree, while CoNLL-U is a flat format. Therefore,

```

docToL0 :: CoNLLU.Doc → [L0.DependencyTree Word]
docToL0 = concatMap sentenceToL0
  where
    sentenceToL0 sentence =
      let idToChildren = MultiMap.fromList do
          word@(_rel → Just rel) ← _words sentence
          return (_head rel, (_deprel rel, word))
          toTree rootId = do
              (dep, child) ← idToChildren ! rootId
              return $
                L0.DependencyTree
                  (toEnum $ fromEnum dep)
                  (toEnum $ fromEnum $ fromMaybe U.X $ _upos child)
                  child $
                    toTree $ _id child
          in toTree $ CoNLLU.SID 0

```

We convert between hs-conllu's and L_0 's representations of UPOS and dependency relations via the integral Enum representation.

```

deriving instance Enum (L0.UPOS w)
deriving instance Enum (L0.Dependency w)

```

3.2. L_1

First, let's get some annoyances out of the way. Since we're leaving all conntation to the human, discourse words are

```

module L1 where

[deflang|
  ((Stripped w) from L0:UD
    (* Dependency
      (- Punct)
      (- Discourse))
    (* UPOS
      (- PUNCT)
      (- X)))]

$(newDeclarationGroup)

[defpass|(from L0:UD to Stripped)]

lower :: L0.DependencyTree w → Maybe (DependencyTree w)
lower = descendDependencyTree xlate

```

```

where
  xlate = Xlate
  { onDependencyDiscourse = Nothing
  , onDependencyPunct = Nothing
  , onUPOSPUNCT = Nothing
  , onUPOSX = Nothing
  , onDependency = const Nothing
  , onDependencyTree = \ (L0.DependencyTree dep pos word children) →
    Just $
      DependencyTree <$>
        descendDependency xlate dep <*>
        descendUPOS xlate pos <*>
        pure word <*>
        pure (mapMaybe (descendDependencyTree xlate) children)
  , onUPOS = const Nothing
  }

```

4. Category theory

4.1. Definitions

If you're already familiar with categories and their representation in Haskell, you can skip this section. (There is already `Control.Category`, but it is necessarily a *wide* category, where its objects are always `Ob(Hask)`.)

Haskell categories

```

class Category (k :: Type → Type → Type) where
  type Ob k (a :: Type) :: Constraint
  id :: Ob k a ⇒ k a a
  (.) :: (Ob k a, Ob k b, Ob k c) ⇒ k b c → k a b → k a c

instance Category (→) where
  type Ob (→) a = ()
  id x = x
  (g . f) x = g (f x)

```

Using `CoNLL-U`

```

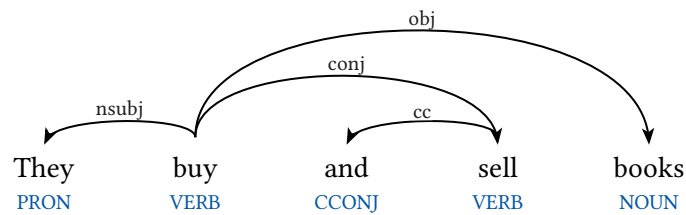
mor(Category, A, A) := id(Category, A).
mor(Category, A, C) := mor(Category, A, B), mor(Category, B, C).

```

4.2. A Dependency Tree Topos

This is the core (Hegelian and category-theoretic) claim: **every noun is a verb-morphism between two other nouns**. For example, Nothing $\xrightarrow{\text{Knowledge}}$ Something.

Consider the dependency tree parse of the following sentence



In prolog

```

upos(word("They", 1), pron).
upos(word("buy", 2), verb).
upos(word("and", 3), cconj).
upos(word("sell", 4), verb).
upos(word("books", 5), noun).

:- table dep/3.

dep(word("buy", 2), [nsubj], word("They", 1)).
dep(word("buy", 2), [conj], word("sell", 4)).
dep(word("buy", 2), [obj], word("books", 5)).
dep(word("sell", 4), [cc], word("and", 3)).

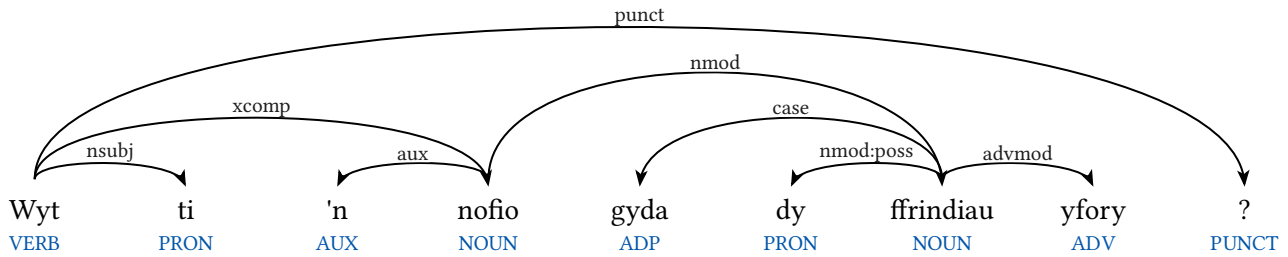
%% Symmetric over conj
dep(Word1, [conj], Word2) :- dep(Word2, [conj], Word1).

%% Semantic meaning of conj
dep(A, Path, Destination) :- dep(A, [conj], B), dep(B, Path, Destination).

%% Category axioms
dep(Word, [], Word) :- Word = word(_, _). % identity
dep(A, Path, C) :- % composition
    dep(A, Subpath1, B),
    dep(B, Subpath2, C),
    append(Subpath1, Subpath2, Path).

link(Noun1, [Verb], Noun2) :-
    upos(Noun1, noun),
    upos(Noun2, noun),
    dep(Verb, [nsubj], Noun1),
    dep(Verb, [obj], Noun2),
    upos(Verb, verb).

%% Category axioms
link(Noun, [], Noun) :- upos(Noun, noun). % identity
link(A, Path, C) :- % composition
    link(A, Subpath1, B),
    link(B, Subpath2, C),
    append(Subpath1, Subpath2, Path).
  
```

Let's define the type of dependency trees.

```

data DependencyTree a = DependencyTree {
  word :: a,
  children :: [(Rel, DependencyTree a)]
} deriving Functor

```

From this, you could construct the free category $\mathcal{D}$ with paths as morphisms. We get the following morphisms in **Cat**:

$$\mathcal{D} \xrightarrow{T} \mathcal{T}$$

To any dependency parse, we can associate a category, say $\mathcal{D}$, by considering the objects as words (and their indices) and the morphisms as dependencies. To be precise, $\text{Ob}(\mathcal{D}) = \{(\text{They}, 1), (\text{buy}, 2), \dots\}$ and $\text{mor}(\mathcal{D}) = \{((\text{buy}, 2), [nsubj], (\text{They}, 1)), \dots\}$. I'll use $\text{buy}_2 \xrightarrow{nsubj} \text{They}_1$ as notation for $\{((\text{buy}, 2), nsubj, (\text{They}, 1))\} \in \text{mor}(\mathcal{D})$ from here forward.

Topos of local truth constructed in just one sentence. First add necessary pushouts of morphisms through relation words to construct just noun phrases as objects in the category. Objects are nominals that are related to

The category of elementary topoi **Top** is a 2-subcategory of the 2-category **Cat**

Maybe– Terminal object: “this”/unrestricted PRON Binary products: different sentences, indicates that two independent clauses can be broken up Equalizer: CCONJ/union Exponential: linguistic recursion/which Omega: “itself” all nouns Sub object classifier: X–copula–>“itself”

Existential quantification would use “this”, whereas the default is just universal. i.e. Red is bright vs this red is bright.

Now that I have a sheaf topos, I would like to be able to glue it together into a general understanding. The glue will inevitably have a nonzero cohomology in which case we will fail to synthesize a global understanding. But say we have a global understanding; then we can understand how different parts fit together by alternating between natural language representation and the categorical.

Once I have a local topos, I can attempt gluing using some kind of ACL2 combo or H-M unification, or both. I will need to figure this out later, and the exact system by which the local, overfit, understanding gets generalized to the global will determine the cohomology.

Maybe have a two-level sheaf attempting to glue sentences together in one argument, using some natural deduction and coreference (H-M unification there?) in the glue between sentences which are part of a topos. Then, can try a different kind of glue to see how two arguments fit together, and which way the functors go between them. Maybe the higher level isn't even a sheaf and just a topology defined on it.

5. Feeling the elephant

6. Diagrams

6.1. 3D

Appendix A. Referenced packages

The following imports were used in this Literate Haskell source file.

```
module Main where

import Prelude hiding (id, (.), read, Word)
import qualified Prelude
import Data.Kind (Constraint, Type)
import Control.Arrow
import Data.Maybe
import Data.MultiMap (MultiMap, (!))
import qualified Data.MultiMap as MultiMap
```

Typst is used for parsing

```
import qualified Typst.Syntax

import System.IO.Unsafe
import qualified Language.C.Inline.Cpp as Cpp
import Foreign.C.String
import Foreign.Marshal.Alloc

import Conllu.Parse
import Conllu.Type (CW(..), Rel(..), AW, Sent(..))
import qualified Conllu.Type as CoNLLU
import qualified Conllu.DeprelTagset as D
import Conllu.UposTagset ()
import qualified Conllu.UposTagset as U
```

Intermediate representations

```
import qualified L0
import qualified L1

type Word = CW AW

Cpp.context Cpp.cppCtx
Cpp.include "<iostream>"
Cpp.include "<sstream>"
Cpp.include "<udpipe.cpp>"
```

Appendix B. Stubs

resolve = undefined
dependencyParse = undefined
understand = undefined
explain = undefined
simplify = undefined
interpret = undefined
execute = undefined
view = undefined
webrender = undefined
createFrame = undefined
read = undefined
initialState = undefined
dialectic = undefined
typstLayout = undefined

data Nominal = Nominal
data Noun = Noun
data Sentence = Sentence
data Text = Text
data Semantics = Semantics
data Math = Math
data Input = Input
data Command = Command
data State = State
data DisplayListItem = DisplayListItem
type DisplayList = [DisplayListItem]
data GLContext = GLContext
data Set a = Set
data Fact = Fact
data Knowledge = Knowledge
data Thesis = Thesis

References

1. Jake. Fanzines from the 1970s. (2015).
2. Voynov, A., Corbi, A., López-Oliver, P. & Gil Oliva, D. Typst: A Modern Typesetting Engine for Science. (2026).
3. De Marneffe, M.-C., Manning, C. D., Nivre, J. & Zeman, D. Universal dependencies. *Computational linguistics* **47**, 255–308 (2021).
4. Keep, A. W. & Dybvig, R. K. A nanopass framework for commercial compiler development. in *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming* 343–350 (2013).